

FOR BUILD AND RELEASE TEAMS

Build receipts for desktop apps

Checked before anything is signed.

This short video explains a new check before desktop apps are signed. No security background is needed.

Today

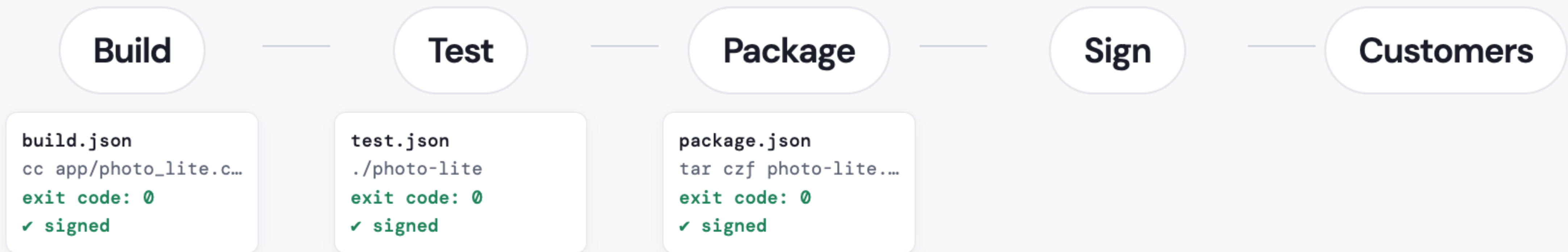


The signing team signs the file it receives.

It cannot see what happened before.

Today, an app is built, tested, packaged, and then signed. The signing team signs the file it receives. But it cannot see what happened in the earlier steps.

Every step writes a **signed receipt**



Every step writes a receipt. A receipt records what ran, what went in, what came out, and the result. The build machine signs each receipt. Any later change is detected.

What changes for your team

● ● ● build-machine

REAL RUN

```
$ cilock run --step build -k ../keys/build-machine.key -o ../evidence/build.json --material-manifest -a  
git -- cc app/photo_lite.c -o photo-lite  
✓ signed receipt saved: evidence/build.json
```

Your build command is unchanged. It goes after the --

Put cilock run in front of each step you already have. Give it the step name, a signing key, and where to save the receipt. Your build command itself does not change.

The release team writes a short **rulebook**

- 1 The build finished without errors.
- 2 The tests ran on that build and passed.
- 3 The package contains the exact file the build produced.

The release team writes a short rulebook. This one has three rules. Rule 1: the build finished without errors. Rule 2: the tests ran on that build and passed. Rule 3: the package contains the exact file the build produced.

● ● ● rulebook/test.rego (excerpt)

RULE 2

```
deny[msg] {  
  has_exit_code  
  input.exitcode != 0  
  msg := sprintf("the tests failed (exit code  
%v)", [input.exitcode])  
}
```

● ● ● rulebook.json (excerpt)

RULE 3

```
"package": {  
  "name": "package",  
  ...  
  "type": "https://aflock.ai/attestations/command-run/v0.2",  
  ...  
  "artifactsFrom": [  
    "build"  
  ]  
}
```

Each rule is a few readable lines. This is the real rule for the tests. The rulebook ties the package to the build output: artifactsFrom. That is rule 3. The release team checks the rulebook and signs it, so nobody can change it quietly.

One check before signing

● ● ● build-machine

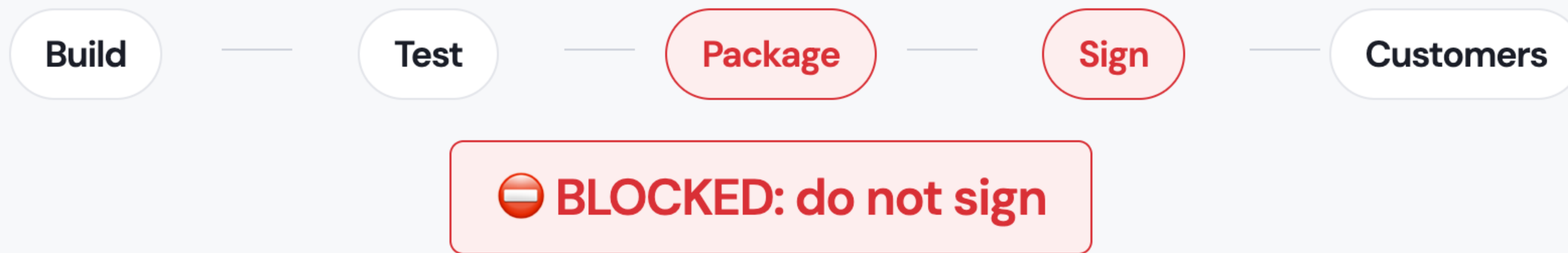
REAL RUN

```
$ cilock verify photo-lite.tar.gz -p ../rulebook/rulebook.signed.json -k ../keys/signing-team.pub -s  
sha1:$(git rev-parse HEAD) -a ../evidence/build.json ...
```

 **PASS: every rule met. OK to sign.**

Before signing, one cilock verify command checks the receipts against the rulebook. If every rule is met, the answer is PASS. Signing continues as normal.

Example 1: the file was replaced



The photo-lite in the package is not the one the build produced.

```
cilock: mismatched digests for photo-lite
```

Someone replaced the app file after the tests ran. The file in the package does not match the build output. Rule 3 fails.

Example 2: the tests were skipped



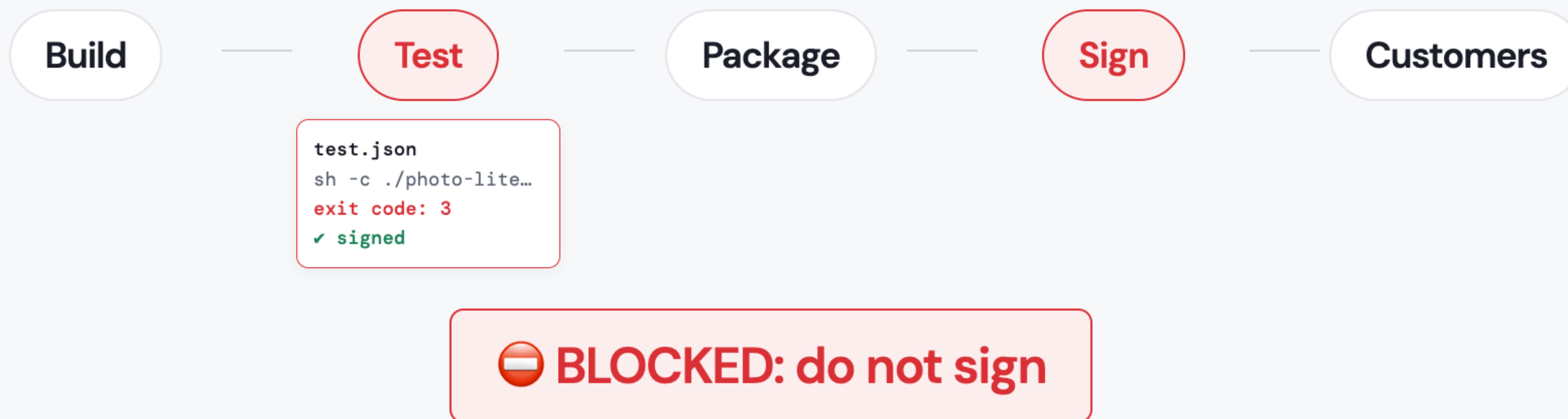
 **BLOCKED: do not sign**

There is no valid receipt for the 'test' step.

`cilock: no collection passed verification for step test`

A rushed release skipped the tests. There is no test receipt, so rule 2 fails. The release is blocked.

Example 3: the tests failed



Rulebook: the tests failed (exit code 3).

```
cilock: policy was denied due to: the tests failed (exit code 3)
```

The tests failed, but the release continued anyway. The test receipt records exit code 3. Rule 2 fails. The release is blocked.

Works in the CI you already have

● ● ● GitHub Actions

REAL EXAMPLE

```
- uses: aflock-ai/cilock-action@v1
  with:
    step: build
    command: "go build -o bin/myapp
./cmd/myapp"
```

● ● ● GitLab CI

REAL TEMPLATE

```
build:
  extends: .cilock
  variables:
    CILOCK_STEP: build
    CILOCK_COMMAND: "go build -o myapp
./cmd/myapp"
```

● ● ● Jenkins

SAME COMMAND

```
sh 'cilock run --step build -k ../keys/build-
machine.key -o ../evidence/build.json --
material-manifest -a git -- cc
app/photo_lite.c -o photo-lite'
```

records CI context from

github

gitlab

jenkins

aws-codebuild

cilock is a pipeline observer. It wraps a step, watches it run, and writes the receipt. It works in GitHub Actions, GitLab CI, Jenkins, or anything that runs a command. It can also record details from the CI system itself.

Built for **AI agents**: a policy tests itself

ROUND	CHANGE	VALIDATOR FINDINGS	VERDICTS RIGHT	TIME
1	first draft	6	3/4	0.3s
2	add artifactsFrom (tie test+package to build)	6	4/4	0.2s
3	fix the guard the validator flagged	0	4/4	0.4s

A policy comes with its own test: known good and known bad builds. Round 1: the first draft. The replaced file wrongly passed. Round 2: all verdicts right, but cilock's validator flags a weak rule. Round 3: zero findings, every verdict right. The agent stops here.

Round 1 → 2: the agent links the steps

rulebook.json · test and package steps

real diff

```
@@ -37,6 +37,9 @@
```

```
    }
```

```
  ]
```

```
}
```

```
+ ],  
+ "artifactsFrom": [  
+   "build"
```

```
]
```

```
},
```

```
"package": {
```

REPLACED FILE, ROUND 1

PASS (should be BLOCKED)

REPLACED FILE, ROUND 2

BLOCKED ✓

In round 1, the replaced file wrongly passed. The agent adds artifactsFrom: test and package must use the exact build output. Now the replaced file is BLOCKED. Four of four verdicts are right.

Round 2 → 3: cilock says how to fix it

```
rulebook/build.rego                                     real diff

@@ -2,4 +2,5 @@

+# Rule 1: the build step ran and finished cleanly.
deny[msg] {
-  not is_number(input.exitcode)
+  not has_exit_code
    msg := "no exit code recorded for the build"
@@ -8,2 +9,3 @@
deny[msg] {
+  has_exit_code
    input.exitcode != 0
@@ -11 +13,5 @@
}

+
+has_exit_code {
+  is_number(input.exitcode)
+}
```

CILOCK VALIDATOR

`not is\_number(input.exitcode)`
never fires when input.exitcode is
missing

FIX SUGGESTED

move the negation into a helper
rule that deny consumes as `not
helper`

VALIDATOR FINDINGS

6 → 0

In round 2, cilock's validator flags a rule that can never fire, and says how to fix it. The agent applies exactly that fix to the rule. Validator findings drop to zero. Each round took under half a second.

What to expect

- 1 A 30 to 60 minute session to walk through your build steps
- 2 We start with one of your pipelines
- 3 Early failures are usually rule details. We adjust the rules with you.

The same lesson passes on Linux, macOS and Windows build machines in CI.
Your own compilers and CI are confirmed during the pilot.

Here is what to expect. First, a 30 to 60 minute session to walk through your build steps. Then we start with one of your pipelines. Early failures are usually rule details. We adjust the rules with you. The lesson passes on Linux, macOS and Windows. Your own tools are confirmed in the pilot.



TESTIFYSEC

Evidence before acceptance.

Try the lesson github.com/aflock-ai/cilock-training

Every command and result in this video comes from a real cilock run.

Thank you. Questions are welcome at the next meeting. Every command and result in this video comes from a real cilock run.